

填空

1. 它们是 运行 态、 就绪 态和阻塞 态。
2. 系统中一个进程由 程序 、 数据集合 和 进程控制块 (PCB) 三部分组成。
3. 进程是一个 动 态概念, 程序是一个 静 态概念。
4. 处于就绪状态的用户进程最多有 4 个, 最少有 0 个。
注意, 题目里给出的是假设当前系统为用户态, 这表明现在有一个进程处于运行状态, 因此最多有 4 个进程处于就绪态。也可能除一个在运行外, 其他 4 个都处于阻塞。这时, 处于就绪的进程一个也没有。
5. 不可剥夺 方式和 剥夺 方式。
6. 进程调度程序具体负责 中央处理机 (CPU) 的分配。
7. 进行作业调度时, 应该注意 CPU 忙碌 作业和 I/O 忙碌 作业的搭配。
8. 操作系统
9. 后备 、 运行 和 完成 三个阶段。
10. 短作业优先 调度算法。

选择

1-5: CADBD, 6-10: CADAC 11: A, D 12: ABCD 13: ABCD 14: AB 15: ABCD 16: ABC
17: ABC 18: ABCD, 19: ABCD, 20: ABCD, 21: C, 22: B, 23: D, 24: C, 25: ABD, 26: A
27: C, 28: A, 29: ABD, 30: D 31: ABCD 32: ABCD 33: C, 34: D 35: C,
36-40: BCDCC, 41-45: BABDD 46-50: AAAEB 51-55: EBCDD 56-60: CBADC 61-63: BCB

三、问答

1. 在多道程序设计系统中, 如何理解“内存中的多个程序的执行过程交织在一起, 大家都在走走停停”这样一个现象?

答: 在多道程序设计系统中, 内存中存放多个程序, 它们以交替的方式使用 CPU。因此, 从宏观上看, 这些程序都开始了自己的工作。但由于 CPU 只有一个, 在任何时刻 CPU 只能执行一个进程程序。所以这些进程程序的执行过程是交织在一起的。也就是说, 从微观上看, 每一个进程一会儿在向前走, 一会儿又停步不前, 处于一种“走走停停”的状态之中。

2. 什么是“原语”、“特权指令”、“系统调用命令”和“访管指令”? 它们之间有无一定的联系?

答: 特权指令和访管指令都是 CPU 指令系统中的指令, 只是前者是一些只能在管态下执行的指令, 后者是一条只能在目态下执行的指令。原语和系统调用命令都是操作系统中的功能程序, 只是前者执行时不能被其他程序所打断, 后者没有这个要求。操作系统中有些系统调用命令是以原语的形式出现的, 例如创建进程就是一条原语式的系统调用命令。但并不是所有系统调用命令都是原语。因为如果那样的话, 整个系统的并发性就不可能得到充分地发挥。

3. 操作系统是如何处理源程序中出现的系统调用命令的?

答: 编译程序总是把源程序中的系统调用命令改写成为一条访管指令和相应的参数。这样在程序实际被执行时, 就通过访管指令进入操作系统, 达到调用操作系统功能子程序的目的。

4. 系统调用与一般的过程调用有什么区别?

答: 系统调用是指在用户程序中调用操作系统提供的功能子程序; 一般的过程调用是指在一个程序中调用另一个程序。因此它们之间有如下三点区别。

(1) 一般的过程调用, 调用者与被调用者都运行在相同的 CPU 状态, 即或都处于目态 (用户程序调用用户程序), 或都处于管态 (系统程序调用系统程序); 但发生系统调用时, 发

出调用命令的调用者运行在目态，而被调用的对象则运行在管态，即调用者与被调用者运行在不同的 CPU 状态。

(2) 一般的过程调用，是直接通过转移指令转向被调用的程序；但发生系统调用时，只能通过访管指令提供的一个统一的入口，由目态进入管态，经分析后，才转向相应的操作系统命令处理程序。

(3) 一般的过程调用，在被调用者执行完后，就径直返回断点继续执行；但系统调用可能会导致进程状态的变化，从而引起系统重新分配处理机。因此，系统调用处理结束后，不一定是返回调用者断点处继续执行。

5. 试述创建进程原语的主要功能。

答：创建进程原语的主要功能有以下三项。

(1) 为新建进程申请一个 PCB。

(2) 将创建者（即父进程）提供的新建进程的信息填入 PCB 中。

(3) 将新建进程设置为就绪状态，并按照所采用的调度算法，把 PCB 排入就绪队列中。

6. 处于阻塞状态的一个进程，它所等待的事件发生时，就把它状态由阻塞改变为就绪，让它到就绪队列里排队，为什么不直接将它投入运行呢？

答：只要是涉及管理，就应该有管理的规则，没有规则就不成方圆。如果处于阻塞状态的一个进程，在它所等待的事件发生时就径直将它投入运行（也就是把 CPU 从当前运行进程的手中抢夺过来），那么系统就无法控制对 CPU 这种资源的管理和使用，进而也就失去了设置操作系统的作用。所以，阻塞状态的进程在它所等待的事件发生时，必须先进入就绪队列，然后再去考虑它使用 CPU 的问题。

7. 作业调度与进程调度有什么区别？

答：作业调度和进程调度（即 CPU 调度）都涉及到 CPU 的分配。但作业调度只是选择参加 CPU 竞争的作业，它并不具体分配 CPU。而进程调度是在作业调度完成选择后的基础上，把 CPU 真正分配给某一个具体的进程使用。

8. 系统中的各种进程队列都是由进程的 PCB 链接而成的。当一个进程的状态从阻塞变为就绪状态时，它的 PCB 从哪个队列移到哪个队列？它所对应的程序也要跟着移来移去吗？为什么？

答：当一个进程的状态从阻塞变为就绪时，它的 PCB 就从原先在的阻塞队列移到就绪队列里。在把进程的 PCB 从这个队列移到另一个队列时，只是移动进程的 PCB，进程所对应的程序是不动的。这是因为在进程的 PCB 里，总是记录有它的程序的断点信息。知道了断点的信息，就能够知道程序当前应该从哪里开始往下执行了。这正是保护现场所起的作用。

9. 为什么说响应比高者优先作业调度算法是对先来先服务以及短作业优先这两种调度算法的折中？

答：先来先服务的作业调度算法，重点考虑的是作业在后备作业队列里的等待时间，因此对短作业不利；短作业优先的作业调度算法，重点考虑的是作业所需的 CPU 时间（当然，这个时间是用户自己估计的），因此对长作业不利。“响应比高者优先”作业调度算法，总是在需要调度时，考虑作业已经等待的时间和所需运行时间之比，即：

$$\text{该作业已等待时间} / \text{该作业所需 CPU 时间}$$

不难看出，这个比值的分母是一个不变的量。随着时间的推移，一个作业的“已等待时间”会不断发生变化，也就是分子在不断地变化。显然，短作业比较容易获得较高的响应比。这是因为它的分母较小，只要稍加等待，整个比值就会很快上升。另一方面，长作业的分母虽然很大，但随着它等待时间的增加，比值也会逐渐上升，从而获得较高的响应比。根据这种分析，可见“响应比高者优先”的作业调度算法，既照顾到了短作业的利益，也照顾到了长作业的利益，是对先来先服务以及短作业优先这两种调度算法的一种折中。

10. 短作业优先调度算法总能得到最小的平均周转时间吗？为什么？

答：短作业优先调度算法只有在所有作业同时到达后备作业队列时，才能得到最小的平均周转时间。如果各作业不是同时到达，这个结论是不成立的。可以用反例说明，例如，教材上举有如下例子：考虑有 5 个作业 A 到 E，运行时间分别是 2、4、1、1、1；到达时间分别是 0、0、3、3、3。按照短作业优先的原则，最初只有 A 和 B 可以参与选择，因为其他 3 个还没有到达。于是，运行顺序应该是 A、B、C、D、E。它们每个的周转时间分别是 2、6、4、5、6，平均周转时间是 4.6。但如果按照顺序 B、C、D、E、A 来调度，它们每一个的周转时间成为 9、4、2、3、4，平均周转时间是 4.4。结果比短作业优先调度算法好。之所以会这样，就是因为这 5 个作业并没有同时到达。

11.

僵尸进程：子进程已终止，但其父进程尚未回收其资源（如进程描述符）。子进程的进程控制块（PCB）仍占用系统资源，处于“僵尸”状态。通常由父进程未调用 `wait()` 或 `waitpid()` 导致。

孤儿进程：父进程终止，子进程仍在运行。孤儿进程会被 `init` 进程（PID 为 1）收养，`init` 会负责清理其资源。通常孤儿进程不会造成问题。

两者区别：僵尸进程占用资源未释放，可能导致资源泄漏；孤儿进程被系统自动管理，无需特别处理。

12.

生产者生成数据，并放入缓冲区；

消费者从缓冲区取出数据，进行处理；

任何时间只有进程访问缓冲区；

13.

在两种情况下会出现循环等待：

缓冲区空时，生产者等待缓冲区的互斥访问，以便放入数据；消费者占有缓冲区访问权，等待生产者放入的数据；

缓冲区满时，生产者占有缓冲区访问权，等待空的缓冲块；消费者等待缓冲区的互斥访问，以便取出数据；

14.

管程是一种并发程序的编程方法，由一个与入口队列对应的锁和若干个与共享数据访问的等待队列对应的条件变量组成，从而实现在任一时刻最多只有一个线程执行管程代码；

15.

相同点：在任一时刻最多只有一个线程执行管程代码或临界区代码；

不同：正在管程中的线程可临时放弃管程的互斥访问，等待事件出现时恢复；而临界区不支持临时退出；

16.

管程允许临时放弃管程的互斥访问，而信号量并不支持临时放弃互斥访问权；

17.

基本状态：执行态：进程正在处理机上运行 就绪态：进程已获得除处理机以外的一切所需资源 阻塞态：进程正在等待某一事件而暂停运行 转换关系：就绪→执行：经处理机调度，就绪进程得到处理机资源 执行→就绪：时间片用完活在可剥夺系统中有更高优先级进程进入 执行→阻塞：进程需要的某一资源还没准备好 阻塞→就绪：进程需要的资源已准备好

18.

死锁是指多个进程因竞争资源而造成的一种僵局(互相等待),若无外力作用,这些进程都无法向前推进

死锁的条件: (1) 互斥条件 (2) 不剥夺条件 (3) 请求并保持条件 (4) 循环等待条件

四、计算

1.

(1) 采用先来先服务作业调度算法时的实施过程如下。

作	业到达时间	所需 CPU 时间	开始时间	完成时间	周转时间
1	0.0	8	0.0	8.0	8.0
2	0.4	4	8.0	12.0	11.6
3	1.0	1	12.0	13.0	12.0

这时，作业的调度顺序是 1→2→3。其平均周转时间为：

$$(8 + 11.6 + 12) / 3 = 10.53$$

(2) 采用短作业优先作业调度算法时的实施过程如下。

作	业到达时间	所需 CPU 时间	开始时间	完成时间	周转时间
1	0.0	8	0.0	8.0	8.0
3	1.0	1	8.0	9.0	8.0
2	0.4	4	9.0	13.0	12.6

这里要注意，在作业 1 运行完毕进行作业调度时，作业 2 和 3 都已经到达。由于是实行短作业优先作业调度算法，因此先调度作业 3 运行，最后调度作业 2 运行。所以，这时的作业调度顺序是 1→3→2。其平均周转时间为：

$$(8 + 8 + 12.6) / 3 = 9.53$$

(3) 还可以有更好的作业调度算法，使其平均周转时间优于这两种调度算法。例如，如果知道在作业 1 后面会来两个短作业，那么作业 1 到达后，先不投入运行。而是等所有作业到齐后，再按照短作业优先作业调度算法进行调度，具体实施过程如下。

作	业到达时间	所需 CPU 时间	开始时间	完成时间	周转时间
3	1.0	1	1.0	2.0	1.0
2	0.4	4	2.0	6.0	5.6
1	0.0	8	6.0	14.0	14.0

这时的作业调度顺序是 3→2→1。其平均周转时间为：

$$(1 + 5.6 + 14) / 3 = 6.87$$

2.

解：（1）采用先来先服务作业调度算法时的实施过程如下。

作业号 到达时间 所需 CPU 时间 开始时间 完成时间 周转时间

1	9:00	70 分钟	9:00	10:10	70 分钟
2	9:40	30 分钟	10:10	10:40	60 分钟
3	9:50	10 分钟	10:40	10:50	60 分钟
4	10:10	5 分钟	10:50	10:55	45 分钟

这时，作业的调度顺序是 1→2→3→4。其平均周转时间为：

$$(70 + 60 + 60 + 45) / 4 = 58.75$$

（2）采用短作业优先作业调度算法时的实施过程如下。

作业号 到达时间 所需 CPU 时间 开始时间 完成时间 周转时间

1	9:00	70 分钟	9:00	10:10	70 分钟
4	10:10	5 分钟	10:10	10:15	5 分钟
3	9:50	10 分钟	10:15	10:25	35 分钟
2	9:40	30 分钟	10:25	10:55	75 分钟

这时，作业的调度顺序是 1→4→3→2。其平均周转时间为：

$$(70 + 5 + 35 + 75) / 4 = 46.25$$

3.

解：（1）采用先来先服务作业调度算法时的实施过程如下。

作业号 到达时间 所需 CPU 时间 开始时间 完成时间 周转时间

1	9:00	70 分钟	9:00	10:10	70 分钟
2	9:40	30 分钟	10:10	10:40	60 分钟
3	9:50	10 分钟	10:40	10:50	60 分钟
4	10:10	5 分钟	10:50	10:55	45 分钟

这时，作业的调度顺序是 1→2→3→4。其平均周转时间为：

$$(70 + 60 + 60 + 45) / 4 = 58.75$$

（2）采用短作业优先作业调度算法时的实施过程如下。

作业号 到达时间 所需 CPU 时间 开始时间 完成时间 周转时间

1	9:00	70 分钟	9:00	10:10	70 分钟
4	10:10	5 分钟	10:10	10:15	5 分钟
3	9:50	10 分钟	10:15	10:25	35 分钟
2	9:40	30 分钟	10:25	10:55	75 分钟

这时，作业的调度顺序是 1→4→3→2。其平均周转时间为：

$$(70 + 5 + 35 + 75) / 4 = 46.25$$

4.

(1) 定义一信号量 S，初始值为 20 。

意义：

$S > 0$ S 的值表示可继续进入售票厅的人数

$S = 0$ 表示售票厅中已有 20 名顾客（购票者）

$S < 0$ $|S|$ 的值为等待进入售票厅的人数

(2) COBEGIN

PROCESS P_i ($i=1, 2, \dots$)

begin

P(S);

进入售票厅;

购票;

退出;

V(S);

end

COEND

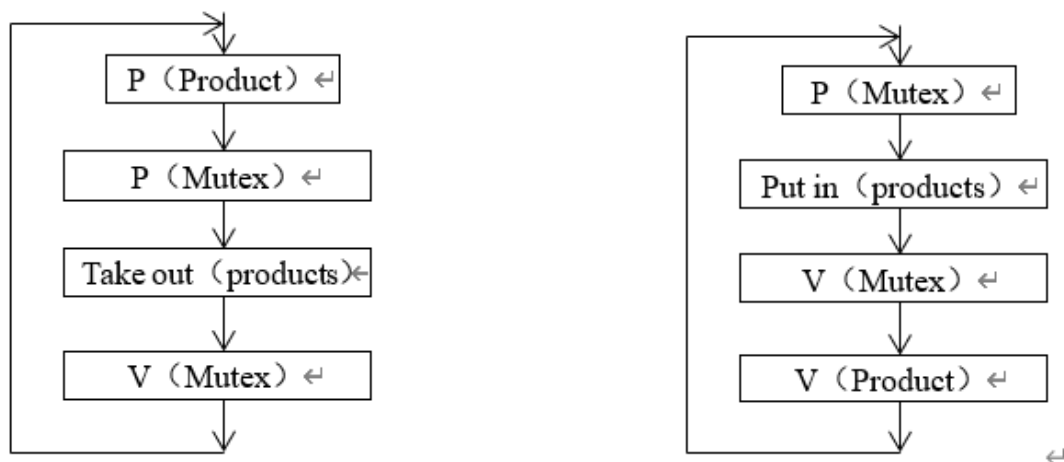
由题每个购票者看作一个进程，所以最多容纳 20 个进程，少于 20 个可以进入，多于 20 个则等待。信号量 $S > 0$ ：可用数目为 $|S|$ ； $S = 0$ ：可用数目为 0，表示已有 20 个进程； $S < 0$ ， $|S|$ 表示有 $|S|$ 个等待进程；

$S--$ ：有一个进程获取资源， $S++$ ：有一个进程释放资源

5.

设置两个信号量为：Product：同步信号量，初值为 0；

Mutex：互斥信号量，初值为 1（仓库为临界资源，描述 a、b 互斥使用仓库）



上图右图：生产 a, b 进程，存放产品 (put in)， $P(\text{Mutex}) = 0$ 资源开始被占用，然后 $V(\text{Mutex})$ 释放资源；产品总量+1，所以 $V(\text{Product})$

左图：消费 c 进程，取出产品 (take out)，产品总量-1，所以 $S(\text{Product})$ ，

同步信号量：同步信号量的值为资源可以使用的个数，信号量小于 0，则线程进行等待，信号量大于 0，表示可用资源个数。初始值 0.

互斥信号量：只有两个值 0 或 1，0 表示资源正在被占用，线程等待。1 表示，资源没有被使用，线程可以进入。初始值为 1

6.

(1) 这一问题是互斥问题。桥是汽车进程互斥使用的资源。(3 分)

(2) 每辆汽车对应一个进程，进程数量不确定。用 P_i ($i=0, 1, 2, \dots$) 表示汽车进程；设互斥信号量 s ，其初值为 1。(2 分)

汽车进程 P_i 的过程可描述如下：

汽车进程 P_i ($i=1, 2, 3, \dots$)

$P(S)$

汽车上桥

在桥上行驶

汽车下桥

$V(S)$

互斥：是指某一资源同时只允许一个访问者对其进行访问，具有唯一性和排它性。但互斥无法限制访问者对资源的访问顺序，即访问是无序的。

同步：是指在互斥的基础上（大多数情况），通过其它机制实现访问者对资源的有序访问。在大多数情况下，同步已经实现了互斥，特别是所有写入资源的情况必定是互斥的。少数情况是指可以允许多个访问者同时访问资源。

7.

(1) semaphore mutex=1 //mutex 为实现进程 A 和进程 C 对共享文件 F 的互斥访问的信号量；

(2) 3 个进程描述如下：

A: {

While(1) {

P(mutex);

读文件 F;

V(mutex);

}

}

```

B: {
While(1) {
    读文件 F;
}

C: {
While(1) {
    P(mutex);
    读文件 F;
    V(mutex);
}
}

```

8.

(1) 如图

Process	Available	Need	Allocation	Available+Available	Finish
	A B C D	A B C D	A B C D	A B C D	
P0	1 6 2 2	0 0 1 2	0 0 3 2	1 6 5 4	T
P3	1 6 5 4	0 6 5 2	0 3 3 2	1 9 8 6	T
P1	1 9 8 6	1 7 5 0	1 0 0 0	2 9 8 6	T
P2	2 9 8 6	2 3 5 6	1 3 5 4	3 12 13 10	T
p4	3 12 13 10	0 6 5 6	0 0 1 4	3 12 14 14	T

存在一个安全序列 {P0, P3, P1, P2, P4}, 不会发生死锁, 所以该状态安全。

(2) 如果分配给 P2, 则如图

Process	Allocation	Need	Available
	A B C D	A B C D	A B C D
P0	0 0 3 2	0 0 1 2	0 4 0 0
P1	1 0 0 0	1 7 5 0	
P2	2 5 7 6	1 1 3 4	
P3	0 3 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

找不到任何一个安全序列，系统不安全所以不能分配给 P2

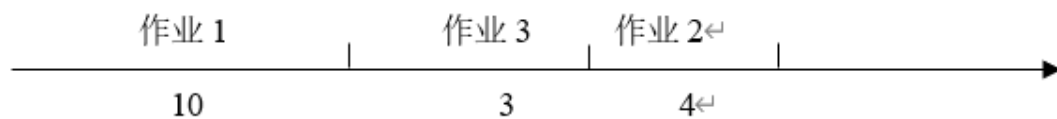
Available:可用资源; Need: 进程所需资源; Allocation: 已分配的资源

只要 $Need \leq Available$ 则可以分配，此进程运行完，释放已分配资源和可用资源，
 Available+Available: 下一次的可用资源

安全序列可能不止一个，只要存在一个，则系统安全

9.

执行时间图：非抢占式优先级算法



Job	进入时刻	运行时间	优先级	开始时刻	结束时刻	周转时间	带权周转时间
1	0	10	2	0	10	10	1
3	2	3	5	10	13	11	11/3
2	1	4	3	13	17	16	4

平均周转： $(10+11+16) / 3 = 37/3$

平均带权周转： $(1+4+11/3) / 3 = 26/9$

10.

解:Wait : -1, signal: +1 所以 $S = S - 9 + 5 = -1$

11.

解:保证不发生死锁,即至少有一个进程能获得所有资源,即至少一个进程能获得2个A资源,其他进程获得一个A资源 所以 系统提供资源数 = 1个A资源*4个进程+1个A资源 = 5

12.

P1() {

 S1;signal(a);signal(b);}

P2() {

 wait(a);S2;signal(c);}

P3() {

 wait(b);S3;signal(d);}

P4() {

 wait(c);wait(d);S4;signal(e);}

P5() {

 wait(e);S5;}

Main() {

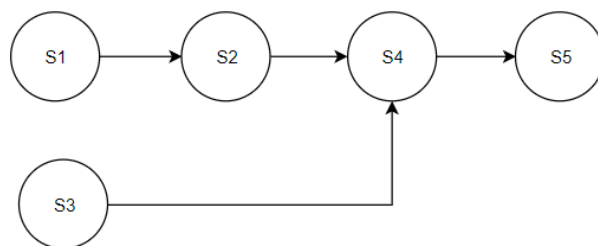
 Semaphore a, b, c, d, e;

 a. value=0, b. value=0, c. value=0, d. value=0, e. value=0;

 cobegin

```
P1 () ; P2 () ; P3 () ; P4 () ; P5 () ;  
  
coend  
  
}
```

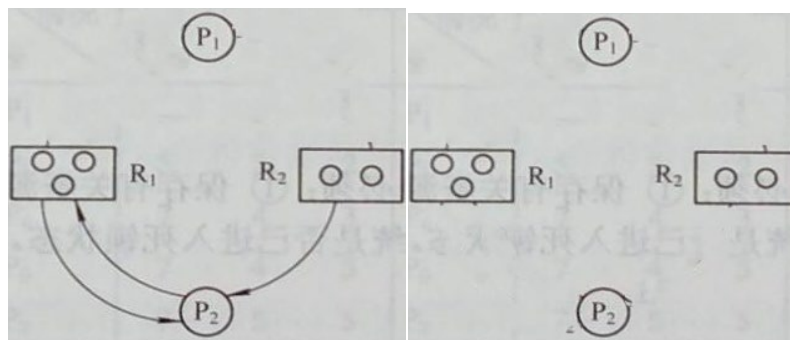
13.



14.

进程 算法		A	B	C	D	E	平均
	到达时间	0	2	4	6	8	——
FCFS	服务时间	4	3	5	2	4	——
	完成时间	4	7	12	14	18	——
	周转时间	4	5	8	8	10	7
SPF	带权周转时间	1	1.67	1.6	4	2.5	2.15
	完成时间	4	7	18	9	13	——
	周转时间	4	5	14	3	5	6.2
	带权周转时间	1	1.67	2.8	1.5	1.25	1.64

15.



R1 分配 2 个资源给 P1, 分配一个资源给 P2, R1 还剩 0 个资源 R2 分配 1 个资源给 P2, R2 还剩 1 个资源 P1 请求 1 个 R2 资源, 可以请求成功 P2 请求 1 个 R1 资源, 不能请求成功 所以先执行 P1 操作, P1 执行完后, 释放资源, 此时 R1 有 2 个资源, R2 有 1 个资源 再执行 P2 操作, P2 请求一个 R1 资源, R1 还剩一个, 执行后释放资源

16.

(1)

进程	Work	Allocation	Need	Work+Allocation
	A B C D	A B C D	A B C D	A B C D
P0	1 6 2 2	0 0 3 2	0 0 1 2	1 6 5 4
P3	1 6 5 4	0 3 3 3	0 6 5 2	1 9 8 7
P4	1 9 8 7	0 0 1 4	0 6 5 6	1 9 9 11
P1	1 9 9 11	1 0 0 0	1 7 5 0	2 9 9 11
P2	2 9 9 11	1 3 5 4	2 3 5 6	3 12 14 15

(2)

因为 $Req(1, 2, 2, 2) < Available(1, 6, 5, 4)$ 且 $Req(1, 2, 2, 2) < Need(2, 3, 5, 6)$ 所以请求是合法的，但是找不到一组安全序列，进程处于不安全状态